

这里的 Θ 称作“大 Θ 记号”（big-theta notation），它是对算法复杂度的准确估计——对于规模为 n 的任何输入，算法的运行时间 $T(n)$ 都与 $\Theta(h(n))$ 同阶。

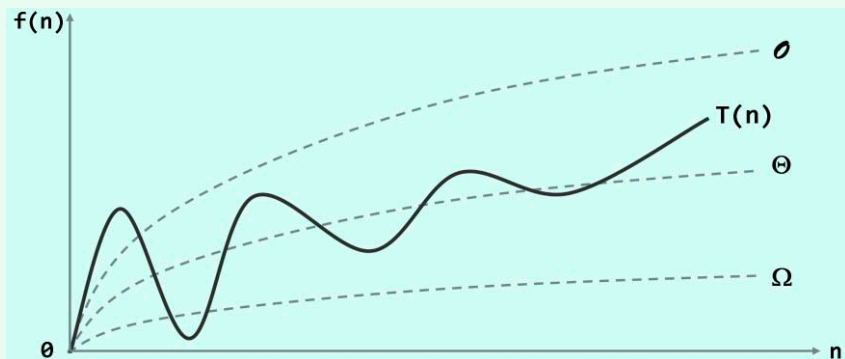


图1.4 大 O 记号、大 Ω 记号和大 Θ 记号

以上主要的这三种渐进复杂度记号之间的联系与区别，可直观地由图1.4示意。

1.2.3 空间复杂度

除了执行时间的长短，算法所需存储空间的多少也是衡量其性能的一个重要方面，此即所谓的空间复杂度（space complexity）。实际上，以上针对时间复杂度所引入的几种渐进记号，也适用于对空间复杂度的度量，其原理及方法基本相同，不再赘述。

需要注意的是，为了更为客观地评价算法性能的优劣，除非特别申明，空间复杂度通常并不计入原始输入本身所占用的空间——对于同一问题，这一指标对任何算法都是相同的。反之，其它（如转储、中转、索引、映射、缓冲等）各个方面所消耗的空间，则都应计入。

另外，很多时候我们都是更多地甚至仅仅关注于算法的时间复杂度，而不必对空间复杂度做专门的考查。这种简便评测方式的依据，来自于以下事实：就渐进复杂度的意义而言，在任一算法的任何一次运行过程中所消耗的存储空间，都不会多于其间所执行基本操作的累计次数。

实际上根据定义，每次基本操作所涉及的存储空间，都不会超过常数规模；纵然每次基本操作所占用或访问的存储空间都是新开辟的，整个算法所需的空间总量，也不过与基本操作的次数同阶。从这个意义上说，时间复杂度本身就是空间复杂度的一个天然的上界。

当然，对空间复杂度的分析也有其自身的意义，尤其在对空间效率非常在乎的应用场合中，或当问题的输入规模极为庞大时，由时间复杂度所确立的平凡上界已经难以令人满意。这类情况下，人们将更为精细地考查不同算法的空间效率，并尽力在此方面不断优化。本书的后续章节，将结合一些实际问题介绍相关的方法与技巧。

§ 1.3 复杂度分析

在明确了算法复杂度的度量标准之后，如何分析具体算法的复杂度呢？1.2.2节所引入的三种记号中，大 O 记号是最基本的，也是最常用到的。从渐进分析的角度，大 O 记号将各算法的复杂度由低到高划分为若干层级级别。以下依次介绍若干典型的复杂度级别，并介绍主要的分析方法与技巧。读者可参照以下介绍的方法，做进一步的练习（习题[1-30]和[1-32]）。

1.3.1 常数 $O(1)$

■ 问题与算法

考查如下常规元素的选取问题，该问题一种解法如算法1.3所示。

```
ordinaryElement(S[], n) //从 $n \geq 3$ 个互异整数中，除最大、最小者以外，任取一个“常规元素”
    任取的三个元素 $x, y, z \in S$ ; //这三个元素亦必互异
    通过比较，对它们做排序; //设经排序后，依次重命名为： $a < b < c$ 
    输出 $b$ ;
```

算法1.3 取非极端元素

该算法的正确性不言而喻，但它需要运行多少时间？与输入的规模 n 有何联系？

■ 复杂度

既然 S 是有限集，故其中的最大、最小元素各有且仅有一个。因此，无论 S 的规模有多大，在任意三个元素中至少都有一个是非极端元素。不妨取前三个元素 $x = S[0]$ 、 $y = S[1]$ 和 $z = S[2]$ ，这一步只需执行三次（从特定单元读取元素的）基本操作，耗费 $O(3)$ 时间。接下来，为确定这三个元素的大小次序，最多需要做三次比较（习题[2-37]），也需 $O(3)$ 时间。最后，输出居中的非极端元素只需 $O(1)$ 时间。因此综合起来，算法1.3的运行时间为：

$$T(n) = O(3) + O(3) + O(1) = O(7) = O(1)$$

运行时间可表示和度量为 $T(n) = O(1)$ 的这一类算法，统称作“常数时间复杂度算法”（constant-time algorithm）。此类算法已是最为理想的，因为不可能奢望“不劳而获”。

一般地，仅含一次或常数次基本操作的算法（如算法1.1和算法1.2）均属此类。此类算法通常不含循环、分支、子程序调用等，但也不能仅凭语法结构的表面形式一概而论（习题[1-7]）。

采用1.2.3节的分析方法不难看出，除了输入数组等参数之外，该算法仅需常数规模的辅助空间。此类仅需 $O(1)$ 辅助空间的算法，亦称作就地算法（in-place algorithm）。

1.3.2 对数 $O(\log n)$

■ 问题与算法

考查如下问题：对于任意非负整数，统计其二进制展开中数位1的总数。

该问题的一个算法可实现如代码1.2所示。

该算法使用一个计数器ones记录数位1的数目，其初始值为0。随后进入一个循环：通过二进制的与（and）运算，检查 n 的二进制展开的最低位，若该位为1则累计至ones。由于每次循环都将 n 的二进制展开右移一位，故整体效果等同于逐个检验所有数位是否为1，该算法的正确性也不难由此得证。

以 $n = 441_{(10)} = 110111001_{(2)}$ 为例，采用以上算法，变量 n 与计数器ones在计算过程中的演变过程如表1.1所示。

表1.1 countOnes(441)的执行过程

十进制	二进制	数位1计数
441	110111001	0
220	110111100	1
110	11011110	1
55	110111	1
27	11011	2
13	1101	3
6	110	4
3	11	4
1	1	5
0	0	6

```

1 int countOnes(unsigned int n) { //统计整数n的二进制展开中数位1的总数:  $O(\log n)$ 
2     int ones = 0; //计数器复位
3     while (0 < n) { //在n缩减至0之前,反复地
4         ones += (1 & n); //检查最低位,若为1则计数
5         n >>= 1; //右移一位
6     }
7     return ones; //返回计数
8 } //等效于glibc的内置函数int __builtin_popcount (unsigned int n)

```

代码1.2 整数二进制展开中数位1总数的统计

■ 复杂度

根据右移运算的性质,每右移一位, n 都至少缩减一半。也就是说,至多经过 $1 + \lfloor \log_2 n \rfloor$ 次循环, n 必然缩减至0,从而算法终止。实际上从另一角度来看, $1 + \lfloor \log_2 n \rfloor$ 恰为 n 二进制展开的总位数,每次循环都将其右移一位,总的循环次数自然也应是 $1 + \lfloor \log_2 n \rfloor$ 。后一解释,也可以从表1.1中 n 的二进制展开一系列清晰地看出。

无论是该循环体之前、之内还是之后,均只涉及常数次(逻辑判断、位与运算、加法、右移等)基本操作。因此,`countOnes()`算法的执行时间主要由循环的次数决定,亦即:

$$O(1 + \lfloor \log_2 n \rfloor) = O(\lfloor \log_2 n \rfloor) = O(\log_2 n)$$

由大 O 记号定义,在用函数 $\log_2 n$ 界定渐进复杂度时,常底数 r 的具体取值无所谓(习题[1-8]),故通常不予专门标出而笼统地记作 $\log n$ 。比如,尽管此处底数为常数2,却可直接记作 $O(\log n)$ 。此类算法称作具有“对数时间复杂度”(logarithmic-time algorithm)。

实际上,代码1.2中的`countOnes()`算法仍有巨大的改进余地(习题[1-12])。

■ 对数多项式复杂度

更一般地,凡运行时间可以表示和度量为 $T(n) = O(\log^c n)$ 形式的这一类算法(其中常数 $c > 0$),均统称作“对数多项式时间复杂度的算法”(polylogarithmic-time algorithm)。上述 $O(\log n)$ 即 $c = 1$ 的特例。此类算法的效率虽不如常数复杂度算法理想,但从多项式的角度看仍能无限接近于后者(习题[1-9]),故也是极为高效的一类算法。

1.3.3 线性 $O(n)$

■ 问题与算法

考查如下问题:计算给定 n 个整数的总和。该问题可由代码1.3中的算法`sumI()`解决。

```

1 int sumI(int A[], int n) { //数组求和算法(迭代版)
2     int sum = 0; //初始化累加器,  $O(1)$ 
3     for (int i = 0; i < n; i++) //对全部共 $O(n)$ 个元素,逐一
4         sum += A[i]; //累计,  $O(1)$ 
5     return sum; //返回累计值,  $O(1)$ 
6 } //  $O(1) + O(n) \cdot O(1) + O(1) = O(n+2) = O(n)$ 

```

代码1.3 数组元素求和算法sumI()

■ 复杂度

`sumI()`算法的正确性一目了然，它需要运行多少时间呢？

首先，对`s`的初始化需要 $O(1)$ 时间。算法的主体部分是一个循环，每一轮循环中只需进行一次累加运算，这属于基本操作，可在 $O(1)$ 时间内完成。每经过一轮循环，都将一个元素累加至`s`，故总共需要做 n 轮循环，于是该算法的运行时间应为：

$$O(1) + O(1) \times n = O(n + 1) = O(n)$$

凡运行时间可以表示和度量为 $T(n) = O(n)$ 形式的这一类算法，均统称作“线性时间复杂度算法”（linear-time algorithm）。比如，算法1.2只需略加修改，即可解决“ n 等分给定线段”问题，这个通用版本相对于输入 n 就是一个线性时间复杂度的算法。

也就是说，对于输入的每一单元，此类算法平均消耗常数时间。就大多数问题而言，在对输入的每一单元均至少访问一次之前，不可能得出解答。以数组求和为例，在尚未得知每一元素的具体数值之前，绝不可能确定其总和。故就此意义而言，此类算法的效率亦足以令人满意。

1.3.4 多项式 $O(\text{polynomial}(n))$

若运行时间可以表示和度量为 $T(n) = O(f(n))$ 的形式，而且 $f(x)$ 为多项式，则对应的算法称作“多项式时间复杂度算法”（polynomial-time algorithm）。比如根据1.2.2节的分析，1.1.3节所实现起泡排序`bubblesort1A()`算法的时间复杂度应为 $T(n) = O(n^2)$ ，故该算法即属于此类。当然，以上所介绍的线性时间复杂度算法，也属于多项式时间复杂度算法的特例，其中线性多项式 $f(n) = n$ 的次数为1。

在算法复杂度理论中，多项式时间复杂度被视作一个具有特殊意义的复杂度级别。多项式级的运行时间成本，在实际应用中一般被认为是可接受的或可忍受的。某问题若存在一个复杂度在此范围以内的算法，则称该问题是可有效求解的或易解的（tractable）。

请注意，这里仅要求多项式的次数为一个正的常数，而并未对其最大取值范围设置任何具体上限，故实际上该复杂度级别涵盖了很大的一类算法。比如，从理论上讲，复杂度分别为 $O(n^2)$ 和 $O(n^{2012})$ 算法都同属此类，尽管二者实际的计算效率有天壤之别。之所以如此，是因为相对于以下的指数级复杂度，二者之间不超过多项式规模的差异只是小巫见大巫。

1.3.5 指数 $O(2^n)$

■ 问题与算法

考查如下问题：在禁止超过1位的移位运算的前提下，对任意非负整数 n ，计算幂 2^n 。

```
1 __int64 power2BF_I(int n) { //幂函数 $2^n$ 算法（蛮力迭代版）， $n \geq 0$ 
2   __int64 pow = 1; //O(1)：累积器初始化为 $2^0$ 
3   while (0 < n --) //O(n)：迭代 $n$ 轮，每轮都
4     pow <<= 1; //O(1)：将累积器翻倍
5   return pow; //O(1)：返回累积器
6 } //O(n) =  $O(2^r)$ ， $r$ 为输入指数 $n$ 的比特位数
```

代码1.4 幂函数算法（蛮力迭代版）